UNIVERSIDAD TORCUATO DI TELLA

TD7: Ingeniería de Datos

Recuperación ante fallas

Tipos de fallas

Tipos de fallas

Los sistemas de bases de datos sufren múltiples tipos de fallas:

- **Fallas de sistema:** Por errores de software o hardware que detienen abruptamente la ejecución del gestor, como las fallas de segmentación, una división por cero, etc.
- **Fallas de dispositivos:** Aquellas que provienen de un daño físico en dispositivos como discos o memoria. Pueden provocar la finalización de un proceso, además de la pérdida de información.
- **Fallas naturales externas:** Provienen de fuera del hardware, como caídas de tensión, terremotos, incendios. Suelen provocar pérdida de información también.

Fallas catastróficas vs. no catastróficas

- En **situaciones catastróficas** como las fallas de dispositivos o las fallas naturales externas, se necesitan **mecanismos de backup** para recuperar la información.
- En cambio, cuando no hay pérdida de información (ej., fallas de sistema), si bien no se necesita acceder a los backups, se debe al menos recuperar a la base de datos para que no quede en un estado inconsistente (garantizar las propiedades ACID).

Mecanismos de recuperación

Mecanismos de recuperación

- Los mecanismos de recuperación son los que permiten volver a la base a un estado consistente en casos de fallas de sistema.
- Estos mecanismos constan de:
  - Algoritmos (implementados por el **gestor de recuperación** del SGBD)
  - Registros de log que el SGBD genera durante su funcionamiento y que serán usados por esos algoritmos.
- Estos mismos registros de log son aprovechados por el SGBD cuando es necesario abortar una transacción por diversos motivos.

El WAL

- El **WAL (*write-ahead log*)** es un archivo que mantiene un registro de todas las operaciones que el gestor realiza sobre la base de datos.
- Almacena los siguientes registros:
  - (BEGIN, T_{id}) → *Comienza la transacción T_{id}*
  - (WRITE, T_{id}, X, v_{old}, v_{new}) → *La transacción T_{id} modifica el ítem X*
  - (READ, T_{id}, X) → *La transacción T_{id} lee el ítem X*
  - (COMMIT, T_{id}) → *Termina exitosamente la transacción T_{id}*
  - (ABORT, T_{id}) → *Se aborta la transacción T_{id}*

El Gestor de Recuperación

- El **Gestor de Recuperación** es un módulo del SGBD que se ocupa de restaurar la base de datos cuando por algún tipo de falla la misma queda en un estado inconsistente. El gestor se guía por dos reglas básicas que hay que cumplir siempre:
  - **Regla WAL (Write Ahead Log):** Esta regla dice que antes de guardar la modificación de un ítem en la base física (en disco) pisando su valor anterior, se debe escribir el registro de *log* correspondiente y asegurarse de que ese registro quede en disco.
  - **Regla FLC (Force Log at Commit):** Cuando una transacción hace commit, el archivo de log debe volcarse a disco (*flush*).

Estrategias de log

- Cumpliendo con las reglas anteriores, existen distintas estrategias posibles de gestión de la recuperación ante fallas:
  - **Método UNDO (actualización inmediata):** Se busca que los datos que una transacción modifica se **guarden** en disco lo antes posible, y obligatoriamente **antes del commit** de la transacción.
  - **Método REDO (actualización diferida):** Se busca que la transacción **no modifique nada en disco hasta que no haya hecho el commit**.
  - **Método UNDO/REDO:** El SGBD tiene la flexibilidad de guardar las modificaciones que hace la transacción en el momento que lo prefiera.

Estrategia UNDO

Estrategia UNDO (actualización inmediata)

- Recordemos que en el método UNDO se busca que los datos que una transacción modifica se guarden en disco lo antes posible, y obligatoriamente antes del commit de la transacción.
- En este caso, si eventualmente hay una falla y la transacción no logra terminar, se usará el log para deshacer todos los cambios que la transacción haya hecho.
- Para ello, se sigue el siguiente procedimiento:

Estrategia UNDO (actualización inmediata)

1. Cuando una transacción modifica un ítem X, escribe un registro en el log indicando cuánto valía ese ítem antes de modificarlo:

   (WRITE, T_i, X, v_{old})

2. Dicho registro de log se vuelca a disco antes de volcar el ítem modificado a disco
3. El ítem modificado se vuelca a disco antes del commit de la transacción.
4. Cuando la transacción termina se hace su commit (es decir, se escribe el registro de (COMMIT, T_i) en disco).

De esta forma, toda transacción commiteada tendrá sus datos a salvo en disco. Y ante una falla previa al commit, todos los datos que la transacción pueda haber modificado en disco se podrán deshacer a través del log.

Recuperación bajo UNDO

Si llegara a producirse una falla, cuando el sistema se reinicia:

1. Recorre el log desde el final hacia atrás, buscando transacciones que no tengan registrado su COMMIT o ABORT.
2. Para cada operación WRITE de una transacción de la que no se encontró el commit, se deshace la escritura colocando el valor v_{old} en dicho item en disco.
3. Cuando se termina de recorrer el log hacia atrás, se escribe un registro de (ABORT, T_i) por cada transacción de la que no se encontró el commit, para darlas por cerradas. Para concluir, se vuelca el log a disco.

¿Y si ocurre una falla durante la recuperación del gestor? Afortunadamente este algoritmo es **idempotente**. Es decir, si se ejecuta parcialmente muchas veces, y finalmente se ejecutara por completo, dejará a la base en un estado consistente, igual que si se hubiera recuperado por completo de una sola vez.

UNDO Logging - Ejemplo

#	Log	T	buffer manager	t	M(A)	M(B)	D(A)	D(B)
1	<START T>						8	8
2			INPUT(A)		**8**		8	8
3		READ(A,t)		**8**	8		8	8
4		t := t*2		**16**	8		8	8
5	<T, A, 8>	WRITE(A,t)		16	**16**		8	8
6			INPUT(B)	16	16	**8**	8	8
7		READ(B,t)		**8**	16	8	8	8
8		t := t*2		**16**	16	8	8	8
9	<T, B, 8>	WRITE(B,t)		16	16	**16**	8	8
10			OUTPUT(A)	16	16	16	**16**	8
11			OUTPUT(B)		16	16	16	**16**
12	<COMMIT T>	COMMIT			16	16	16	16

- ¿Qué pasa si hay una falla tras el paso ....?
  - 12: No hay que hacer nada, la base quedó consistente
  - 11: La transacción quedó incompleta, deshacer los cambios
    - Escribir 8 para A y B en el disco, agregar < ABORT T > al log y bajarlo al disco

Estrategia REDO

Estrategia REDO (actualización diferida)

- En este caso se busca que la transacción no modifique nada en disco hasta que no haya hecho el commit.
- De esta forma, si hay una falla durante la ejecución de la transacción, no hay que preocuparse por nada de lo que hizo.
- En cambio, podría ocurrir que debido a una falla inmediatamente posterior al commit, no se haya llegado a volcar a disco los cambios que la transacción había hecho, y el gestor de recuperación tendrá que usar el log para rehacerlos.

Estrategia REDO (actualización diferida)

El procedimiento bajo REDO es el siguiente:

1. Cuando una transacción T_i modifica un ítem X, escribe un registro en el log indicando el nuevo valor de ese ítem:

   (WRITE, T_i, X, v_{new})

2. Cuando la transacción termina se hace su commit. Es decir, se escribe el registro de (COMMIT, T_i) en disco.

3. En algún momento posterior, el SGBD decidirá volcar ese ítem a disco

De esta forma, una transacción no commiteada jamás habrá enviado datos a disco. Entonces, ante una falla previa al commit no habrá nada que hacer. Pero ante una falla posterior al COMMIT, podría ser necesario escribir en disco los datos que la transacción modificó y que no se llegaron a guardar.

Recuperación bajo REDO

Si llegara a producirse una falla, cuando el sistema se reinicia:

1. Recorre el log desde el final hacia atrás, buscando transacciones que sí tengan registrado su COMMIT.
2. Luego, desde el inicio del log hacia adelante, para cada operación WRITE de una transacción commiteada, el gestor de recuperación rehace la escritura, asegurando que el valor que había escrito la transacción se vuelque en disco.
3. Cuando se termina de recorrer el log hacia adelante, se escribe un registro de (ABORT, T_i) por cada transacción de la que no se encontró el commit, para darlas por cerradas. Para concluir, se vuelca el log a disco.

El algoritmo REDO nos plantea un nuevo problema. Según este procedimiento, cuando hay una falla se deberían rehacer transacciones que quizás fueron commiteadas años atrás. Es necesario contar con “marcas” que nos digan que hasta cierto punto todas las transacciones están aseguradas en disco.

REDO Logging - Ejemplo

#	Log	T	buffer manager	t	M(A)	M(B)	D(A)	D(B)
1	<START T>						8	8
2			INPUT(A)		**8**		8	8
3		READ(A,t)		**8**	8		8	8
4		t := t*2		**16**	8		8	8
5	<T, A, 16>	WRITE(A,t)		16	**16**		8	8
6			INPUT(B)	16	16	**8**	8	8
7		READ(B,t)		**8**	16	8	8	8
8		t := t*2		**16**	16	8	8	8
9	<T, B, 16>	WRITE(B,t)		16	16	**16**	8	8
10	<COMMIT T>	COMMIT			16	16	8	8
11			OUTPUT(A)		16	16	**16**	8
12			OUTPUT(B)		16	16	16	**16**

- ¿Qué pasa si hay una falla tras el paso ....?
  - 11: Rehacer: Escribir 16 para A y B en el disco
    - Los cambios no se bajaron a disco pero la transacción comiteó
  - 12: Ídem anterior
    - En este caso, la base quedó consistente pero podría no ser así...

Puntos de control (checkpoints) en REDO

Cada cierto tiempo el gestor volcará a disco los ítems en memoria y escribirá una marca en el log que indique que desde ese punto hacia atrás todo está en disco.

El procedimiento para construir este **checkpoint** es el siguiente:

1. Se escribe un registro (BEGIN CKPT, t_{act}) en el log, registrando todas las transacciones que están activas en ese instante.
2. Se van volcando a disco todos los ítems modificados por transacciones commiteadas antes del BEGIN CKPT.
3. Al terminar, se escribe un registro de (END CKPT) en el log y se vuelca el log a disco.

Puntos de control (checkpoints) en REDO

Este tipo de checkpoint se denomina **fuzzy checkpoint**, porque evita detener el procesamiento de transacciones durante su construcción.

Ejercicio

Ejercicio

Un SGBD implementa el algoritmo de recuperación REDO con checkpoint activo. Luego de una falla, el sistema encuentra el siguiente archivo de log: Los datos en el disco se deben hacer después del commit.

```
01 (BEGIN, T1);
02 (WRITE T1, X, 8);
03 (WRITE T1, Y, 3);
04 (BEGIN, T2);
05 (WRITE T2, Z, 2);
06 (WRITE T2, W, 4);
07 (COMMIT, T2);
08 (BEGIN CKPT, T1);
09 (BEGIN, T3);
10 (WRITE T3, W, 1);
11 (COMMIT, T1);
12 (END CKPT);
13 (WRITE T3, X, 2);
```

Asegura la escritura de todo lo anterior (T2 por commit)

Sí hay commit de T1, se rehace por las dudas.

No hay commit de T3.

¿Cómo se llevará a cabo el procedimiento de recuperación? Indicar qué cambios deben ser realizados en disco y en el archivo de log.

Ejercicio: respuesta

La transacción T2 estará garantizada en disco. Se deberá rehacer la transacción T1 y deshacer la transacción T3.
Por lo tanto, hay que escribir 8 en X y 3 en Y, y volcar estos datos en disco. Luego hay que escribir (ABORT, T3) en el log, y volcar el log a disco.